

Práctico AYED1 - Sala 2 - 28-09

Práctico 3 - Ejercicio 10

Derivar:

$$g.xs = \langle \text{Max } p, q: 0 \leq p < q < \#xs : xs!p + xs!q \rangle$$

Ejemplo:

$g.[-10, 3, 6, -2]$?

- Rango: $p, q \in \{ (0, 1), (0, 2), (0, 3),$
- $(1, 2), (1, 3),$
- $(2, 3) \}$ ($n * (n-1) / 2$ donde $n = \#xs$)

$$g.[-10, 3, 6, -2] = (xs!0 + xs!1) \max \dots \max (xs!1 + xs!2) \max \dots \max (xs!2 + xs!3) \\ = 3 + 6 = 9$$

Derivación: Por inducción en xs .

Caso base: $xs = []$

$g.[]$

$= \{ \text{esp.} \}$

$\langle \text{Max } p, q: 0 \leq p < q < \#[] : []!p + []!q \rangle$

$= \{ \text{def. } \# \}$

$\langle \text{Max } p, q: 0 \leq p < q < 0 : []!p + []!q \rangle$

= { rango vacío }
 - ∞

Paso inductivo: $xs = y \blacktriangleright ys$

H.I. : $g.ys = \langle \text{Max } p, q: 0 \leq p < q < \#ys : ys!p + ys!q \rangle$

$g.(y \blacktriangleright ys)$
 = { esp. }
 $\langle \text{Max } p, q: 0 \leq p < q < \#(y \blacktriangleright ys) : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle$
 = { def. # }
 $\langle \text{Max } p, q: 0 \leq p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle$
 = { separamos la 1er desigualdad }
 $\langle \text{Max } p, q: \underline{0 \leq p} \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle$
 = { algebra }
 $\langle \text{Max } p, q: (p = 0 \vee 1 \leq p) \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle$
 = { distributividad }
 $\langle \text{Max } p, q: (p = 0 \wedge p < q < \#ys + 1) \vee (1 \leq p \wedge p < q < \#ys + 1) : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle$
 = { part. de rango, y junto la desigualdad en el 2do cuantificador. }
 $\langle \text{Max } p, q: p = 0 \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle_{\text{max}}$
 $\langle \text{Max } p, q: \underline{1 \leq p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q} \rangle$
 = { dos cambios de variable $p \rightarrow p + 1, q \rightarrow q + 1$ }
 $\langle \text{Max } p, q: p = 0 \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle_{\text{max}}$
 $\langle \text{Max } p, q: \underline{1 \leq p + 1 < q + 1 < \#ys + 1} : (y \blacktriangleright ys)!(p + 1) + (y \blacktriangleright ys)!(q + 1) \rangle$
 = { resto 1 en los términos de la desigualdad }
 $\langle \text{Max } p, q: p = 0 \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle_{\text{max}}$
 $\langle \text{Max } p, q: 0 \leq p < q < \#ys : (y \blacktriangleright ys)!(p + 1) + (y \blacktriangleright ys)!(q + 1) \rangle$
 = { def. de ! sabiendo que p y q son ≥ 0 }

$\langle \text{Max } p, q: p = 0 \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle \text{max}$
 $\langle \text{Max } p, q: 0 \leq p < q < \#ys : ys!p + ys!q \rangle$
 $= \{ \text{H.I.} \}$
 $\langle \text{Max } p, q: \underline{p = 0} \wedge p < q < \#ys + 1 : (y \blacktriangleright ys)!p + (y \blacktriangleright ys)!q \rangle \text{max } g.ys$
 $= \{ \text{eliminación de variable } p \}$
 $\langle \text{Max } q: 0 < q < \#ys + 1 : \underline{(y \blacktriangleright ys)!0} + (y \blacktriangleright ys)!q \rangle \text{max } g.ys$
 $= \{ \text{def !} \}$
 $\langle \text{Max } q: 0 < q < \#ys + 1 : y + (y \blacktriangleright ys)!q \rangle \text{max } g.ys$
 $= \{ \text{acá podríamos modularizar pero tratemos de simplificar un poco más.}$
 $\quad \text{cambiamos } 0 < q \text{ por } 1 \leq q. \}$
 $\langle \text{Max } q: 1 \leq q < \#ys + 1 : y + (y \blacktriangleright ys)!q \rangle \text{max } g.ys$
 $= \{ \text{cambio de variable } q \rightarrow q+1 \}$
 $\langle \text{Max } q: \underline{1 \leq q+1 < \#ys + 1} : y + \underline{(y \blacktriangleright ys)!(q+1)} \rangle \text{max } g.ys$
 $= \{ \text{resto 1 en rango, def ! en termino} \}$
 $\langle \text{Max } q: 0 \leq q < \#ys : y + ys!q \rangle \text{max } g.ys$
 $= \{ \text{acá también podríamos modularizar, pero recordamos que } + \text{ distribuye con max}$
 $\quad \text{(asumiremos que el neutro de max es absorbente de la } + : (-\infty) + x = -\infty \text{) } \}$
 $\quad (\langle \text{Max } q: 0 \leq q < \#ys : \underline{ys!q} \rangle + y) \text{max } g.ys$
 $= \{ \text{ahora sí modularizamos, es el máximo de una lista:}$
 $\quad \text{maxLista.xs} = \langle \text{Max } q: 0 \leq q < \#xs : xs!q \rangle \}$
 $\quad (\text{maxLista.ys} + y) \text{max } g.ys$

Ejercicio: derivar maxLista.

Resultado final:

$g.[] \doteq -\infty$

$g.(x \blacktriangleright xs) \doteq (\text{maxLista.xs} + x) \text{max } g.xs$

$\text{maxLista}[] \doteq -\infty$

$\text{maxLista}(x \blacktriangleright xs) \doteq x \max \text{maxLista}.xs$

Ejemplo de ejecución: al maxLista no lo haremos paso por paso

$g.[-10, 3, 6, -2] \rightarrow (\text{maxLista}.[3, 6, -2] + (-10)) \max g.[3, 6, -2]$
 $\rightarrow (6 + (-10)) \max g.[3, 6, -2]$
 $\rightarrow (6 + (-10)) \max (\text{maxLista}.[6, -2] + 3) \max g.[6, -2]$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max g.[6, -2]$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max (\text{maxLista}.[-2] + 6) \max g.[-2]$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max (-2 + 6) \max g.[-2]$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max (-2 + 6) \max g.[-2]$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max (-2 + 6) \max (\text{maxLista}[] + (-2)) \max g.[]$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max (-2 + 6) \max (-\infty + (-2)) \max (-\infty)$
 $\rightarrow (6 + (-10)) \max (6 + 3) \max (-2 + 6) \max (-\infty) \max (-\infty)$
 $\dots$
 $\rightarrow (-4) \max 9 \max 4 \max (-\infty) \max (-\infty)$
 $\rightarrow 9$

Práctico 3 - Ejercicio 7a

Especificar y derivar semiEco que me dice si una palabra empieza con eco.

Ejemplos: "paparulo", "ananá", ...

Pregunta:

- $\text{semiEco}.$ "mmondiola" ? quedamos en que sí.

- semiEco."hhh" ? sí

Especificación:

$\text{semiEco.xs} = \langle \exists \text{ as, bs, cs} : \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} : \text{as} \neq [] \wedge \text{as} = \text{bs} \rangle$

Observación: por regla de intercambio podemos pasar libremente los predicados entre el rango y el término.

Otra opción:

$\text{semiEco.xs} = \langle \exists \text{ as, bs} : \text{xs} = \text{as} ++ \text{as} ++ \text{bs} : \text{as} \neq [] \rangle$

Derivación (con la 2da opción): Por inducción en xs.

Caso base: $\text{xs} = []$

$\text{semiEco.}[]$
 $= \{ \text{esp.} \}$
 $\langle \exists \text{ as, bs} : \underline{[] = \text{as} ++ \text{as} ++ \text{bs}} : \text{as} \neq [] \rangle$
 $= \{ \text{prop. listas} \}$
 $\langle \exists \text{ as, bs} : \text{as} = [] \wedge \text{bs} = [] : \text{as} \neq [] \rangle$
 $= \{ \text{elim. var. as} \}$
 $\langle \exists \text{ bs} : \text{bs} = [] : [] \neq [] \rangle$
 $= \{ \text{rango unitario} \}$
 $[] \neq []$
 $= \{ \text{lógica} \}$
 False

Paso inductivo: $\text{xs} = y \blacktriangleright \text{ys}$

H.I. : $\text{semiEco.y} = \langle \exists \text{ as, bs} : \text{ys} = \text{as} ++ \text{as} ++ \text{bs} : \text{as} \neq [] \rangle$

$\text{semiEco.}(y \blacktriangleright ys)$
 $= \{ \text{esp.} \}$
 $\langle \exists as, bs : y \blacktriangleright ys = as ++ as ++ bs : as \neq [] \rangle$
 $= \{ \text{regla de intercambio (intro algo)} \}$
 $\langle \exists as, bs : y \blacktriangleright ys = as ++ as ++ bs \wedge as \neq [] : \text{True} \rangle$
 $= \{ \text{cambio de variable } as \rightarrow a \blacktriangleright as \text{ (es biyectivo ya que } as \neq []) \}$
 $\langle \exists a, as, bs : y \blacktriangleright ys = (a \blacktriangleright as) ++ (a \blacktriangleright as) ++ bs \wedge a \blacktriangleright as \neq [] : \text{True} \rangle$
 $= \{ \text{prop. listas y def } ++ \}$
 $\langle \exists a, as, bs : y \blacktriangleright ys = a \blacktriangleright (as ++ (a \blacktriangleright as) ++ bs) : \text{True} \rangle$
 $= \{ \text{prop. listas} \}$
 $\langle \exists a, as, bs : y = a \wedge ys = as ++ (a \blacktriangleright as) ++ bs : \text{True} \rangle$
 $= \{ \text{eliminación de variable } a \}$
 $\langle \exists as, bs : ys = as ++ (y \blacktriangleright as) ++ bs : \text{True} \rangle$
 $= \{ \text{para poder aplicar H.I. tengo que hacer aparecer de nuevo el "as \neq []".}$

usamos 3ro excluido para esto. lo podría haber hecho antes quizás,
 pero **nunca** antes del cambio de variable porque justamente habla
 de la nueva as que introduje con el cambio. }

 $\langle \exists as, bs : ys = as ++ (y \blacktriangleright as) ++ bs \wedge (as \neq [] \vee as = []) : \text{True} \rangle$
 $= \{ \text{distributividad y partición de rango} \}$
 $\langle \exists as, bs : ys = as ++ (y \blacktriangleright as) ++ bs \wedge as = [] : \text{True} \rangle \vee$
 $\langle \exists as, bs : ys = as ++ (y \blacktriangleright as) ++ bs \wedge as \neq [] : \text{True} \rangle$
 $= \{ \text{trabajamos el de abajo para buscar la H.I., usamos intercambio} \}$
 $\langle \exists as, bs : ys = as ++ (y \blacktriangleright as) ++ bs \wedge as = [] : \text{True} \rangle \vee$
 $\langle \exists as, bs : ys = as ++ (y \blacktriangleright as) ++ bs : as \neq [] \rangle$

Acá nos trabamos porque nos queda jodiendo la " $y \blacktriangleright$ ".

Debemos generalizar. Pregunta: ¿Sirve esta generalización?

$$\text{genSemiEco.xs.z} = \langle \exists \text{ as, bs : xs = as ++ (z \blacktriangleright \text{as}) ++ bs : as \neq [] \rangle$$

No, porque genSemiEco no generaliza a semiEco porque no puedo elegir ningún z tal que

$$\text{genSemiEco.xs.z} = \text{semiEco.xs}$$

Directamente no es una generalización.

Necesitamos un pasito más:

$$\langle \exists \text{ as, bs : ys = as ++ (y \blacktriangleright \text{as}) ++ bs \wedge \text{as} = [] : \text{True} \rangle \vee$$

$$\langle \exists \text{ as, bs : ys = as ++ (y \blacktriangleright \text{as}) ++ bs : \text{as} \neq [] \rangle$$

$$= \{ \text{prop. listas} \}$$

$$\langle \exists \text{ as, bs : ys = as ++ (y \blacktriangleright \text{as}) ++ bs \wedge \text{as} = [] : \text{True} \rangle \vee$$

$$\langle \exists \text{ as, bs : ys = as ++ } \underline{[y]} \text{ ++ as ++ bs : as} \neq [] \rangle$$

Y acá generalizamos agregando un parámetro para reemplazar a “[y]”:

$$\text{genSemiEco.xs.zs} = \langle \exists \text{ as, bs : xs = as ++ zs ++ as ++ bs : as} \neq [] \rangle$$

Ahora sí, genSemiEco generaliza a semiEco, ya que

$$\text{genSemiEco.xs.[]} = \text{semiEco.xs.}$$

Demostración de que genSemiEco generaliza a semiEco:

$$\text{genSemiEco.xs.[]} =$$

$$= \{ \text{esp. genSemiEco} \}$$

$$\langle \exists \text{ as, bs : xs = as ++ [] ++ as ++ bs : as} \neq [] \rangle$$

$$= \{ \text{prop. listas} \}$$

$$\langle \exists \text{ as, bs : xs = as ++ as ++ bs : as} \neq [] \rangle$$

$$= \{ \text{esp. semiEco} \}$$

semiEco.xs

Luego, si miramos de abajo para arriba esta demostración, funciona como una derivación de semiEco que no usa inducción y que termina en algo que es parte del lenguaje de programación ya que es una llamada a una función. Así que tenemos el siguiente resultado parcial:

semiEco : [Char] → Bool
semiEco.xs ≐ genSemiEco.xs.[]

Observación: Como siempre que hacemos generalización, el intento de derivación por inducción de semiEco se descarta, sólo sirvió para darnos cuenta de que debemos generalizar.

Sólo falta derivar genSemiEco. Ejercicio que deben traer resuelto para otro día.

Especificación alternativa usando “eco” del ejercicio anterior:

semiEco.xs = $\langle \exists \text{ as, bs} : \text{xs} = \text{as} ++ \text{bs} : \text{eco.as} \rangle$

NOTA APARTE (Y REPASO):

$0 \leq q < \#ys$	<u>no es equivalente a</u>	$q = 0 \vee 1 \leq q < \#ys$
$0 \leq q < \#ys + 1$	<u>sí es equivalente a</u>	$q = 0 \vee 1 \leq q < \#ys + 1$

